

# Normalizing Batch Normalization for Long-Tailed Recognition

Yuxiang Bao<sup>1</sup>, Guoliang Kang<sup>2</sup>, Linlin Yang<sup>3</sup>, Xiaoyue Duan, Bo Zhao,  
and Baochang Zhang<sup>4</sup>, *Senior Member, IEEE*

**Abstract**—In real-world scenarios, the number of training samples across classes usually subjects to a long-tailed distribution. The conventionally trained network may achieve unexpected inferior performance on the rare class compared to the frequent class. Most previous works attempt to rectify the network bias from the data-level or from the classifier-level. Differently, in this paper, we identify that the bias towards the frequent class may be encoded into features, i.e., the rare-specific features which play a key role in discriminating the rare class are much weaker than the frequent-specific features. Based on such an observation, we introduce a simple yet effective approach, normalizing the parameters of Batch Normalization (BN) layer to explicitly rectify the feature bias. To achieve this end, we represent the Weight/Bias parameters of a BN layer as a vector, normalize it into a unit one and multiply the unit vector by a scalar learnable parameter. Through decoupling the direction and magnitude of parameters in BN layer to learn, the Weight/Bias exhibits a more balanced distribution and thus the strength of features becomes more even. Extensive experiments on various long-tailed recognition benchmarks (i.e., CIFAR-10/100-LT, ImageNet-LT and iNaturalist 2018) show that our method outperforms previous state-of-the-arts remarkably.

**Index Terms**—Long-tailed recognition, batch normalization, deep learning.

Received 19 December 2023; revised 24 June 2024 and 7 November 2024; accepted 28 November 2024. Date of publication 27 December 2024; date of current version 13 January 2025. This work was supported in part by the National Key Research and Development Program of China under Grant 2023YFC3306401 and Grant 2022YFB3104900, in part by the National Natural Science Foundation of China under Grant 62406298, in part by Zhejiang Provincial Natural Science Foundation under Grant LD24F020007, in part by Beijing Natural Science Foundation under Grant L223024 and Grant L244043, and in part by the “One Thousand Plan” Projects in Jiangxi Province under Grant Jxsq2023102268. An earlier version of this paper was presented at the IEEE Publication Technology Group, Piscataway, NJ [DOI: 10.1109/TIP.2024.3518099]. The associate editor coordinating the review of this article and approving it for publication was Dr. Ioannis Pratikakis. (Yuxiang Bao and Guoliang Kang contributed equally to this work.) (Corresponding author: Baochang Zhang.)

Yuxiang Bao, Guoliang Kang, and Xiaoyue Duan are with the School of Automation Science and Electrical Engineering, Beihang University, Beijing 100191, China (e-mail: yxbao@buaa.edu.cn; kgl.pml@gmail.com; LittleMoon@buaa.edu.cn).

Linlin Yang is with the State Key Laboratory of Media Convergence and Communication, Communication University of China, Beijing 100024, China (e-mail: lyang@cuc.edu.cn).

Bo Zhao is with the School of Artificial Intelligence, Shanghai Jiao Tong University, Shanghai 200240, China (e-mail: bo.zhao@sjtu.edu.cn).

Baochang Zhang is with the School of Artificial Intelligence, Beihang University, Beijing 100191, China, also with the Hangzhou Research Institute, Beihang University, Hangzhou 310051, China, and also with Nanchang Institute of Technology, Nanchang 330044, China (e-mail: bezhang@buaa.edu.cn).

This article has supplementary downloadable material available at <https://doi.org/10.1109/TIP.2024.3518099>, provided by the authors.

Digital Object Identifier 10.1109/TIP.2024.3518099

## I. INTRODUCTION

RECENT years have witnessed a rising of computer vision applications, which greatly benefits from large-scale datasets, e.g., ImageNet [1], COCO [2]. These datasets are usually balanced, i.e., the number of samples across different categories are comparable. However, in real world, the distribution of the number of samples across different classes may be long-tailed, which means the number of samples from “frequent” classes may be much larger than those from “rare” classes. Consequently, the conventional training process may be biased towards frequent classes, making the accuracy on the rare classes not satisfactory. As in practice we usually expect the model to achieve equally good performance on each class, a series of recognition methods [3], [4], [5], [6] are designed for the long-tailed setting, aiming to deal with the above issue.

The key of long-tailed recognition problem lies in how to rectify the network bias towards the frequent classes. Previous works mainly rectify the bias either from the data-level (e.g., data rebalancing [8], [9], data augmentation [10], [11]) or the classifier-level (classifier-decouple training [5], loss function designs [12], [13]). Despite remarkable progress achieved by those works, they don’t explicitly rectify the bias in features which may largely determine the generalization ability of models. Consequently, the feature bias resulted by long-tailed distribution cannot be effectively reduced and thus largely restricts the improvement of long-tailed recognition performance.

From a high-level, different feature channels may represent different characteristics of the image. We may evaluate the importance of a feature channel to a specific class via the classifier weight connecting corresponding feature and class. Feature channels which are important for both classes indicates the characteristics which are shared by both classes. As the characteristics shared by two classes may not help to distinguish samples, we visualize the feature channels which are uniquely important for scoring one specific class. As shown in Fig. 1, the “Fountain Pen” is a rare class while the “Eraser” is a frequent class. Given an image of “Fountain Pen”, the model incorrectly discriminates it into the “Eraser” class. To understand what features the model relies on to distinguish a “Fountain Pen” from an “Eraser”, we visualize attentions [7] of feature channels which are uniquely important for scoring “Fountain Pen” (the second column of attention maps), and those which are uniquely important for scoring

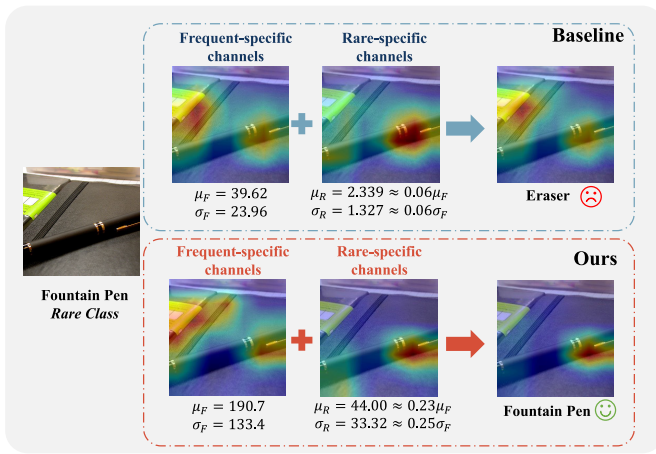


Fig. 1. Illustration of feature bias with a rare class sample. We visualize the attention maps [7] with respect to feature channels which are uniquely important for scoring “fountain pen” (the second column of attention maps) and “eraser” (the first column of attention maps) respectively. Besides, we visualize the attention maps of all feature channels. The statistics of feature channels, including mean  $\mu$ , and standard deviation  $\sigma$ , are calculated with samples in the balanced test set to represent the strength of features. For baseline, though the rare-specific features emerge, their weak strength makes the overall attention unexpectedly focus on irrelevant regions. In contrast, our method strengthens those rare-specific features, enabling the model to focus on class-discriminative regions. See more details in our supplementary materials.

“Eraser” (the first column of attention maps). We also visualize the attention of all feature channels which largely correlates with the final classification. We observe that the features that uniquely represent “Fountain Pen” do exist, but they may not dominate the classification result as the strength of them are significantly weaker than the features that uniquely represent the “Eraser”. That means, in conventional training, the features which are important for correctly classifying samples into rare classes are largely under-estimated. Thus, the overall distribution of the strength of features is strongly biased towards the frequent class (see Fig. 6), rendering it hard to make correct classifications on the rare class.

In this paper, we propose to rectify the feature bias via a simple operation, *i.e.*, normalizing the parameters of Batch Normalization layer. As we know, Batch Normalization (BN) layer is a *de facto* recipe for the conventional deep convolutional networks. It typically follows the convolutional layer or fully-connected layer to adjust the statistics of features to reduce the covariate shift. Forwarding through BN, each feature can be roughly viewed as a Gaussian variable with the mean  $\beta$  (the Bias parameter of BN) and the standard deviation  $\gamma$  (the Weight parameter of BN). Larger Weight and Bias means the feature channel is statistically stronger. Thus, one possible way to rectify the feature bias (*i.e.*, to make the strengths of different feature channels more balanced distributed) is to make the distribution of Weight and Bias parameters of a BN layer more balanced. Our strategy is simple: we treat the Weight parameters in one BN layer as a vector, normalize such a vector into a unit one, and multiply the unit vector by a scalar learnable parameter. For the Bias parameters of BN layer, we operate in the same way. In this manner, we decouple the direction and magnitude of the BN

Weight/Bias vector to learn, which encourages more balanced distribution of mean and variance of features (see Fig. 1) and thus rectifies the feature bias.

We conduct experiments on various long-tailed recognition benchmarks including CIFAR-10/100-LT [3], [14], ImageNet-LT [15] and iNaturalist 2018 [16]. The experiment results show that our method performs favourably against previous state-of-the-arts. In contrast to some previous methods which are carefully designed to strike a balance between the accuracy of frequent and rare classes, we empirically find our method remarkably improves the accuracy of rare classes without sacrificing the accuracy of frequent classes. We also demonstrate that our operation is plug-and-play and can be combined with other state-of-the-art approaches (*e.g.*, classifier-bias rectification methods) to improve the accuracy further.

In a word, our contributions are summarized as follows

- We observe that without explicit rectification, the learned features may be strongly biased towards the frequent classes and its distribution may exhibit unexpected imbalance, *i.e.*, the features, which are specifically important for discriminating rare classes, are weak. Consequently, the statistically strong frequent features (which are representatives for the frequent class but not for the rare class) may drive the model to classify the sample into the frequent class, despite that the sample belongs to a rare one.
- Based on our observations, we introduce a simple yet effective strategy to explicitly rectify the feature bias in the long-tailed scenario, *i.e.*, normalizing the parameters of BN layer. Through decoupling the direction and magnitude of parameters of the BN layer, our method yields more balanced parameters of the BN layer and thus reduces the bias in features.
- We extensively demonstrate the effectiveness of our method on various representative long-tailed image recognition benchmarks, *i.e.*, Long-tailed CIFAR-10/100 [3], ImageNet-LT [15] and iNaturalist 2018 [16]. We demonstrate that our method is plug-and-play and brings consistent improvements compared to previous methods, achieving new state-of-the-arts. Ablations and analyses verify the necessity and effectiveness of each design.

## II. RELATED WORK

### A. Long-Tailed Recognition

Existing strategies for long-tailed recognition can be roughly categorized into three groups: data balancing [5], [10], [11], [17], [18], [19], loss function design [3], [12], [13], [20], and model ensembling [21], [22]. Data balancing can be further divided into data re-sampling and data re-weighting. It is intuitive to re-sample the input data to ensure the balance of a training batch. Existing data re-sampling methods either over-sample the rare cases [23], [24], [25], [26] or generate the tail samples [11], [27] or using data augmentation techniques [28], [29], [30], [31], like mixup [28] and cutout [29]. Existing data re-weighting methods assign

weights to different classes [4] or even to each instance [19], where the weight should be inversely proportional to its frequency in the training set. Instead of data balancing, another line of works is to design specific loss functions to benefit the rare-class classification. Those works usually encourage large margins between rare and frequent classes [3], [12], [13], [32]. Recently, model ensemble strategies [21], [22] introduce multi-expert architectures that learn diverse classifiers in parallel and make the final decision by averaging the predictions of all the experts. As discussed, most previous works relieve the bias from data-level or classifier-level, not explicitly rectifying the bias that exists in the feature. In this paper, we aim to explicitly deal with the feature bias to improve the long-tailed recognition performance.

### B. Batch Normalization

Batch normalization (BN) [33] has been widely used as a basic normalization technique in the majority of state-of-the-art architectures. Specifically, it normalizes the activation value with the statistics of a mini-batch data to meet the standard normal distribution and then applies a linear transformation to the standardized value. Previous works [27], [34], [35], [36] have modified the BN statistics to improve the model's domain adaptation or long-tailed learning ability. For example, CBN [36] designs a novel compound batch normalization strategy to alleviate the long-tailed issue. In this paper, we aim to rectify the feature bias via normalizing BN parameters in the long-tailed recognition setting.

## III. METHODS

### A. Batch Normalization Revisiting

Batch normalization (BN) is one of the basic components in advanced convolutional neural networks. Generally speaking, it adopts mini-batch statistics (*i.e.*, mean and standard deviation) to standardize each feature, and then applies an affine transformation with learnable parameters, *i.e.*, Weight ( $\gamma$ ) and Bias ( $\beta$ ) to the standardized feature. Formally, the operation of BN can be represented as

$$y_k = \gamma_k \hat{x}_k + \beta_k \quad \text{where} \quad \hat{x}_k = \frac{x_k - \mu_k}{\sigma_k}, \quad (1)$$

where  $x_k$  denotes the feature of the  $k$ -th channel. The  $\mu_k$  and  $\sigma_k$  denote the mean and standard deviation of the  $k$ -th channel feature respectively. With  $\mu_k$  and  $\sigma_k$ , the  $x_k$  is standardized to  $\hat{x}_k$  that subjects to a standard normal distribution, *i.e.*,  $\mathcal{N}(0, 1)$ . During training,  $\mu_k$  and  $\sigma_k$  are estimated online with a mini-batch of data, and during testing, we usually adopt the moving average of  $\mu_k$  and  $\sigma_k$  estimated at each training iteration to perform feature standardization.

To maintain the representation capacity of the network, the affine transformation with learnable parameters (*i.e.*, the Weight vector  $\gamma$  and the Bias vector  $\beta$ ) are introduced to recover the statistics of features. In Eq. 1,  $\gamma_k$  and  $\beta_k$  denotes the  $k$ -th elements of  $\gamma$  and  $\beta$  respectively. Therefore, the  $k$ -th feature can be roughly viewed as a random variable which subjects to a normal distribution  $\mathcal{N}(\beta_k, \gamma_k^2)$ .

For long-tailed recognition,  $\gamma$  and  $\beta$  are updated by samples from both frequent and rare classes. Thus, the features, which are representatives for a rare class but not shared by frequent classes, may not be fully trained, *i.e.*, the mean and variance of such "rare-specific" features are under-estimated and relatively small. This characteristic renders the features inherently biased towards frequent classes, which may degenerate the classification accuracy on rare classes.

### B. Normalizing Batch Normalization

As discussed above, the features, which are exclusively important for discriminating rare classes, may be unexpectedly weakened during the long-tailed training. Such weakness of feature can be reflected by the small Weight ( $\gamma_k$ ) and Bias ( $\beta_k$ ) of corresponding BN (as illustrated in Fig. 1). Thus, a straightforward way to reduce such bias in features is to impose a regularization term that minimizes the variance of parameters in a BN layer. However, though such a regularization way can encourage more balanced Weight/Bias parameters of a BN layer, it is task-agnostic and may impair the model capacity. In practice, it is hard to strike a balance between fitting the training data and balancing the parameters of BN layer, which yields sub-optimal results.

In this paper, we propose to normalize the parameters of BN layer (NBN) to rectify the feature bias, without modifying the training objective or adding additional regularization term.

Specifically, we view the Weight/Bias parameters of a BN layer as a vector and decouple the learning of its magnitude and direction. Such an operation is formulated as

$$\mathbf{y} = g_\gamma \frac{\tilde{\gamma}}{\|\tilde{\gamma}\|} \circ \hat{\mathbf{x}} + g_\beta \frac{\tilde{\beta}}{\|\tilde{\beta}\|}, \quad (2)$$

where  $\mathbf{y} = [y_1, y_2, \dots, y_K]$  and  $\hat{\mathbf{x}} = [\hat{x}_1, \hat{x}_2, \dots, \hat{x}_K]$ . The Weight and Bias vector are decoupled:  $g_\gamma$  and  $g_\beta$  are learnable parameters which represent the magnitudes, and the  $\tilde{\gamma}/\|\tilde{\gamma}\|$ , and  $\tilde{\beta}/\|\tilde{\beta}\|$  are the unit vectors, which represent the directions. The  $\circ$  denotes the element-wise product.

1) *Why Can NBN Yield More Balanced Weight/Bias Vector?*: We take the updating of Weight vector of the BN layer as an example to illustrate NBN's balancing effect. In the process of back-propagation, the gradient of the loss function  $\mathcal{L}$  with respect to  $\tilde{\gamma}_k$  (the  $k$ -th element of  $\tilde{\gamma}$ ) is

$$\nabla_{\tilde{\gamma}_k} \mathcal{L} = \frac{g_\gamma}{\|\tilde{\gamma}\|} \left( \nabla_{\gamma_k} \mathcal{L} + \alpha \frac{\tilde{\gamma}_k}{\|\tilde{\gamma}\|} \right), \quad (3)$$

where  $\nabla_{\gamma_k} \mathcal{L}$  refers to the gradient with respect to  $\gamma_k$ , and  $\alpha = -\nabla_{g_\gamma} \mathcal{L}$  means the negative value of gradient with respect to the magnitude  $g_\gamma$ .

Through Eq. 3, we can analyze the relationship between the gradient with respect to the normalized Weight  $\tilde{\gamma}_k$  (*i.e.*,  $\nabla_{\tilde{\gamma}_k} \mathcal{L}$ ) and the gradient with respect to the original Weight  $\gamma_k$  (*i.e.*,  $\nabla_{\gamma_k} \mathcal{L}$ ). As in Eq. 3,  $\frac{g_\gamma}{\|\tilde{\gamma}\|}$  is just a scaling factor which is shared across different channels. Thus, we focus on the term  $R = \alpha \frac{\tilde{\gamma}_k}{\|\tilde{\gamma}\|}$  to analyze the rectification effects on different channels. Considering different effects of  $R$ , there are two different patterns for updating the parameters of the BN layer: (A) when  $\alpha > 0$  and (B) when  $\alpha < 0$ .

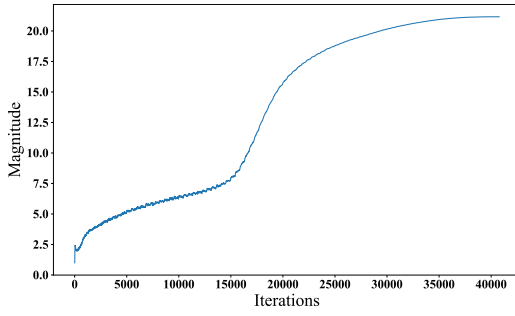


Fig. 2. Visualization of the magnitude  $g_\gamma$  during the training process. The results come from experiments on ImageNet-LT. The trends are similar on other datasets.

In pattern (A), as  $\alpha = -\nabla_{g_\gamma} \mathcal{L} > 0$ ,  $g_\gamma$  will be increased. According to the gradient descent update rule *i.e.*,  $\tilde{\gamma}_k \leftarrow \tilde{\gamma}_k - \epsilon \nabla_{\tilde{\gamma}_k} \mathcal{L}$  where  $\epsilon$  is the learning rate,  $R$  generally penalizes the magnitude of  $|\tilde{\gamma}_k|$ , which means introducing  $R$  encourages  $|\tilde{\gamma}_k|$  to be smaller. However, the intensity of penalties varies across channels. The larger  $|\tilde{\gamma}_k|/\|\tilde{\gamma}\|$  leads to a stronger penalization on  $|\tilde{\gamma}_k|$ . Thus, in this pattern,  $|\tilde{\gamma}_k|$  is encouraged to be more evenly distributed.

In pattern (B), as  $\alpha = -\nabla_{g_\gamma} \mathcal{L} < 0$ ,  $g_\gamma$  will be decreased. Meanwhile, as  $g_\gamma \frac{|\tilde{\gamma}_k|}{\|\tilde{\gamma}\|}$  reflects the variance  $\sigma_k$  of  $k$ -th channel, the decreasing of  $g_\gamma$  results in the decreasing of  $\sigma_k$ , which may impair the ability to fit the attributes with large variance (especially for frequent-specific features). According to the gradient descent update rule,  $R$  generally increases the magnitude of  $|\tilde{\gamma}_k|$ , where larger  $|\tilde{\gamma}_k|/\|\tilde{\gamma}\|$  encourages larger  $|\tilde{\gamma}_k|$ . Thus, in pattern (B), the model tends to increase  $|\tilde{\gamma}_k|$  to compensate for the decreasing of  $g_\gamma$  to avoid capacity loss and encourage data-fitting.

To summarize, the model can adaptively switch between fitting data (pattern (B)) and balancing the parameters of the BN layer (pattern (A)). As shown in Fig. 2,  $g_\gamma$  generally increases during training, which means the term  $\alpha = -\nabla_{g_\gamma} \mathcal{L}$  usually satisfies  $\alpha > 0$  (pattern (A)). It may be because increasing  $g_\gamma$  is much easier and more effective to increase the feature variance  $\sigma$ . Consequently, the parameters of BN become more evenly distributed without disturbing the data-fitting.

2) *Connection With Variance Regularization:* As discussed above, one straightforward way to encourage more evenly distributed Weight/Bias parameters of BN layer is to additionally impose a regularization term that penalizes the variance of Weight/Bias parameters. In terms of the balancing effect, our NBN shares some similarities with such a variance regularization way. However, there exist some key differences between NBN and the variance regularization way: 1) Our NBN doesn't modify the training objective, while variance regularization imposes a strong prior which may conflict with the training objective. As discussed, NBN can adaptively switch between the data-fitting process and the parameter balancing process. As a result, NBN encourages more balanced parameters of BN layer, while not disturbing the fitting to training data. In contrast, too strong variance regularization may impair the model capacity and degenerate the model's performance. 2) Our NBN doesn't introduce

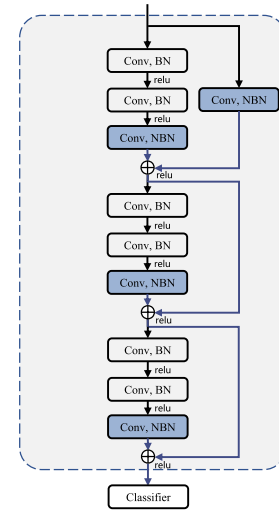


Fig. 3. Illustration of positions to insert the normalizing batch normalization (NBN) layer in the ResNet architecture. We only insert NBN to the last stage of ResNet architecture which consists of three sequential residual blocks.

additional hyper-parameters, while for variance regularization, we need to carefully set a hyper-parameter to strike a balance between optimizing the training objective and penalizing the variance of parameters.

3) *Connection With Weight Normalization [37]:* From the operation level, we find that previous Weight Normalization (WN) work [37] shares a similar idea to ours, *i.e.*, decoupling the magnitude and direction of convolutional weights to learn. However, our method is different from WN in nature, as 1) The motivation is different. We aim to rectify the feature bias existing in the long-tailed recognition setting. The NBN is just a simple and effective way to rectify the feature bias in a deep neural network. In contrast, the WN method aims to ease network optimization and accelerate the training. It is originally employed in a network without any BN layers. 2) The type of layers where the normalization is applied is different and the consequent effect is different. Our NBN is applied to the parameters of BN layer, while the WN method is applied to the parameters of convolutional layers. Thus, our NBN may yield more balanced parameters of BN, while the WN method brings faster convergence. Empirically, we find that applying normalization the same way as WN has no effect on improving the long-tailed recognition performance (see Sec. Ablation Studies).

### C. NBN in ResNet

In this section, we discuss how to insert NBN into the representative ResNet architecture. We aim to explicitly rectify the bias of features which are the output of the last residual block and directly fed into the classifier to make predictions. Meanwhile, through rectification of the last-layer features, we expect that the feature bias from shallow layers may also be rectified through gradient back-propagation.

As shown in Fig. 3, we replace the original BN layers with our designed NBN layers at specific locations of the last stage of ResNet which consists of three residual blocks. Through residual connections, the last-layer features are a summation of

outputs from three residual blocks. Thus, we choose to replace the last BN layers of three residual blocks and the BN layer in the first down-sampling residual path with our proposed NBN layers, as illustrated in Fig. 3. In this way, all sources of bias injected into the last-layer features can be effectively rectified, which yields more evenly distributed strength (reflected by corresponding mean and variance) of features. We empirically verify the effectiveness of our choice in Sec. Ablation Studies.

#### D. Logits Rectification

In addition to the feature bias, the classifier output also contains bias. Specifically, the model trained with samples imbalanced distributed across classes tends to allocate higher confidence to the frequent class. It is because the statistics (*i.e.*, mean and variance) of logits from frequent classes are inherently larger than those from rare classes, rendering the classifier biased towards the frequent classes. Thus, we propose to rectify such bias existing in the classifier via a simple Gaussian standardization operation, *i.e.*, for each logit,

$$\hat{z} = \frac{z - \mu}{\sigma}, \quad (4)$$

where  $z$  denotes the logit. The  $\sigma$  and  $\mu$  denote the standard deviation and mean of the logit. Inspired by BN, during training, we adopt mini-batch statistics of the logit as  $\mu$  and  $\sigma$ , and for testing, we use the moving average of statistics at each iteration to represent  $\mu$  and  $\sigma$ .

### IV. EXPERIMENTS

#### A. Datasets

We test our method on widely used long-tailed recognition benchmarks including CIFAR-10-LT, CIFAR-100-LT [3], [14], ImageNet-LT [15] and iNaturalist 2018 [16]. Following general practice [4], [38], we employ imbalance factor  $N_{max}/N_{min}$  to depict the extent of imbalance in the specific dataset, where  $N_{max}$  and  $N_{min}$  denote the number of samples in most and least frequent classes respectively.

1) *CIFAR-10/100-LT*: The original CIFAR-10 (CIFAR-100) dataset [14] consists of 50,000 training images and 10,000 test images of size  $32 \times 32$ , which are uniformly distributed among 10 (100) classes. We follow [4] to manually create the long-tailed version of CIFAR-10 and CIFAR-100 by randomly removing a certain amount of training examples. The imbalance factor of the created dataset is 100.

2) *ImageNet-LT*: We follow [15] to construct the ImageNet-LT dataset from the original ImageNet dataset [1]. The constructed dataset consists of 115.8K training examples belonging to 1,000 classes. The imbalance factor is 256, with the numbers of the most and least frequent classes at 1280 and 5 respectively.

3) *iNaturalist2018*: The iNaturalist2018 [16] is real-world fine-grained visual recognition datasets. It has naturally long-tailed distributions. And it consists of 435,712 training samples within 8,142 classes. The imbalance factor is 500, with the numbers of the most and least frequent classes at 1000 and 2 respectively.

TABLE I  
ACCURACY ON CIFAR-10-LT AND CIFAR-100-LT WITH  
THE IMBALANCE FACTORS OF 50, 100, AND 200

| Dataset                 | CIFAR-10-LT |             |             | CIFAR-100-LT |             |             |
|-------------------------|-------------|-------------|-------------|--------------|-------------|-------------|
| <i>Imbalance Ratio</i>  | 200         | 100         | 50          | 200          | 100         | 50          |
| Cross Entropy           | 74.0        | 78.0        | 82.3        | 41.2         | 46.1        | 52.0        |
| Focal Loss [19]         | 74.1        | 77.1        | 83.0        | 41.5         | 46.3        | 52.0        |
| LDAM Loss [3]           | 75.0        | 78.9        | 84.3        | 42.3         | 48.3        | 53.3        |
| decoupling LWS [5]      | 78.1        | 83.7        | 84.7        | 47.5         | 53.3        | 58.7        |
| Balanced Softmax [13]   | 79.8        | 83.6        | 86.5        | 46.8         | 51.7        | 57.6        |
| RIDE [21]               | 81.5        | 85.1        | 88.7        | <b>51.8</b>  | 57.3        | 61.7        |
| Cross Entropy + Ours    | 77.5        | 81.5        | 86.3        | 46.3         | 50.5        | 56.9        |
| Balanced Softmax + Ours | 80.2        | 84.1        | 87.0        | 50.5         | 53.7        | 58.9        |
| RIDE + Ours             | <b>84.5</b> | <b>87.1</b> | <b>89.7</b> | <b>51.8</b>  | <b>57.5</b> | <b>62.2</b> |

TABLE II  
ACCURACY ON IMAGENET-LT

|                         | All         | Tail        | Medium      | Head        |
|-------------------------|-------------|-------------|-------------|-------------|
| Cross Entropy           | 45.8        | 9.0         | 39.0        | 67.2        |
| decoupling cRT [5]      | 49.6        | 27.4        | 46.2        | 61.8        |
| decoupling LWS [5]      | 49.9        | 30.3        | 47.2        | 60.2        |
| Balanced Softmax [13]   | 51.1        | 30.2        | 48.1        | 62.3        |
| MiSLAS [39]             | 52.7        | 35.8        | 51.3        | 61.7        |
| PaCo [39]               | 56.0        | 33.7        | 55.7        | 64.4        |
| ACE [40]                | 56.6        | -           | -           | -           |
| ALA [41]                | 52.4        | 35.7        | 49.1        | 62.4        |
| xERM [42]               | 54.1        | 27.5        | 50.0        | <b>68.6</b> |
| TSC [43]                | 54.2        | -           | -           | -           |
| RIDE [21]               | 56.8        | 36.0        | 53.8        | 68.2        |
| SuperDisco [44]         | 57.1        | 37.1        | 53.3        | 66.1        |
| BCL [45]                | 56.7        | 36.5        | 53.9        | 67.2        |
| RIDE + MJB [46]         | 57.7        | 37.7        | 54.1        | 68.4        |
| ABC-Norm [47]           | 51.7        | 33.1        | 49.7        | 60.7        |
| IIF [48]                | 52.8        | 18.9        | 47.0        | 72.1        |
| LogN [49]               | 51.6        | 35.0        | 50.3        | 59.1        |
| CBN [36]                | 57.4        | -           | -           | -           |
| Cross Entropy + Ours    | 49.4        | 16.2        | 45.2        | 66.4        |
| Balanced Softmax + Ours | 53.8        | 33.4        | 50.9        | 64.6        |
| RIDE + Ours             | <b>58.2</b> | 36.7        | <b>56.0</b> | <b>68.6</b> |
| BCL + Ours              | 57.7        | <b>38.5</b> | 55.3        | 67.5        |

#### B. Implementation Details

For CIFAR-10-LT, we adopt ResNet-32 [50] as our backbone. Following [13], the models are trained for 13,000 iterations with a cosine learning rate schedule, and the warm-up iteration number is set as 800. We adopt the same data augmentation strategies as [13], including Cutout [29] and auto-augmentation. For CIFAR-100-LT, we make some modifications to ResNet-32, making the channels of each layer four times the width of the original version. We make such a modification as we empirically find this generally yields better long-tailed recognition performance for both the baseline methods and our method. For ImageNet-LT, following [5], [21], [45], we utilize ResNeXt-50 [51] as the backbone in the main comparison and ResNet-50 [50] for ablation studies. The training lasts 90/100 epochs for most of our experiments. For the purpose of comparing with the state-

TABLE III  
ACCURACY ON iNATURALIST 2018

|                             | All         | Tail        | Medium      | Head        |
|-----------------------------|-------------|-------------|-------------|-------------|
| Cross Entropy               | 65.2        | 60.0        | 67.2        | 75.8        |
| decoupling cRT [5]          | 68.2        | 66.1        | 68.8        | 73.2        |
| Balanced Softmax [13]       | 69.8        | 69.7        | 70.1        | 69.2        |
| DisAlign [20]               | 70.2        | 69.4        | 71.3        | 68.0        |
| MiSLAS [27]                 | 71.6        | 70.4        | 72.4        | 73.2        |
| Balanced Softmax + CMO [11] | 70.9        | 72.3        | 70.0        | 68.8        |
| LTR-WD [6]                  | 70.2        | 69.7        | 70.4        | 71.2        |
| GCL [32]                    | 71.0        | 71.5        | 71.3        | 67.5        |
| BCL [45]                    | 71.7        | 71.6        | 72.0        | 70.9        |
| TSC [43]                    | 70.3        | -           | -           | -           |
| SuperDisco [44]             | 73.6        | 71.3        | 72.9        | 72.3        |
| MBJ [46]                    | 73.2        | -           | -           | -           |
| RIDE [21]                   | 74.8        | 74.7        | 75.0        | 74.5        |
| ABC-Norm [47]               | 71.4        | 70.4        | 73.2        | 68.1        |
| DTRG [52]                   | 69.5        | -           | -           | -           |
| CBN [36]                    | 74.8        | -           | -           | -           |
| Cross Entropy + Ours        | 66.5        | 61.2        | 68.5        | <b>76.6</b> |
| Balanced Softmax + Ours     | 71.6        | 71.3        | 71.7        | 72.1        |
| BCL + Ours                  | 72.6        | 72.3        | 73.2        | 70.8        |
| RIDE + Ours                 | <b>75.3</b> | <b>75.1</b> | <b>75.4</b> | 75.5        |

TABLE IV  
COMPARISON TO VARIANCE REGULARIZATION WITH ITS STRENGTH COEFFICIENT  $\tilde{\alpha}$  AT 0.1 AND 1. THE EXPERIMENTS ARE CONDUCTED ON IMAGENET-LT AND RESNET-50 IS ADOPTED

|                                                    | All         | Tail        | Medium      | Head        |
|----------------------------------------------------|-------------|-------------|-------------|-------------|
| Cross Entropy                                      | 42.2        | 6.3         | 34.7        | 64.2        |
| Variance regularization ( $\tilde{\alpha} = 0.1$ ) | 44.9        | 10.5        | 38.1        | 65.4        |
| Variance regularization ( $\tilde{\alpha} = 1$ )   | 43.3        | 7.0         | 36.1        | 65.0        |
| NBN                                                | <b>47.3</b> | <b>13.4</b> | <b>40.9</b> | <b>67.1</b> |

of-the-art numbers, we cite the number of [44] and [43] with 200 and 300 training epochs respectively. For [39], we report the result of 180 epochs cited from [45]. For iNaturalist, the experiment setup is similar to ImageNet-LT except that all the experiments are conducted with ResNet-50 trained for 200 epochs. For the experiments combining our method with the multi-expert model RIDE [21], we follow the same setup as RIDE and adopt its four-expert version for ImageNet-LT and its three-expert version for CIFAR-10/100-LT and iNaturalist.

In all the experiments, we adopt the SGD optimizer with momentum 0.9 to optimize the model. We empirically find that sharing the magnitude parameter between the Weight and Bias vector or not achieves comparable results. Thus, for simplicity, we use the same magnitude parameter in NBN, *i.e.*, setting  $g = g_\gamma = g_\beta$ . All the models are initialized with random weights. For each model, we train it three times and report the average result. For a better comparison with previous works, besides the overall accuracy, we additionally report the accuracy with respect to three groups [15] of classes: Tail group (classes containing less than 20 training samples), Medium group (classes containing 20 to 100 training samples), and Head group (classes containing more than 100 training samples), on ImageNet-LT and iNaturalist 2018 datasets.

### C. Comparison With State-of-the-Art Methods

1) *Baselines*: To verify the effectiveness of our method, we make comparisons with previous state-of-the-art methods. The “Cross Entropy” means the baseline method that trains the model with cross-entropy loss and doesn’t consider the long-tailed distribution of training samples. The other methods are all the long-tailed recognition methods. To show our method is plug-and-play, we combine our method with previous typical long-tailed recognition methods, *i.e.*, Balanced Softmax [13] which is a loss function specifically designed for the long-tailed setting, RIDE [21] which is an ensemble method with multiple experts, and BCL [45] which applies supervised contrastive learning to the long-tailed scenario. To ensure a fair comparison, we run the officially released code of those works under a unified setting and compare them. We also compare our method with two-stage ones, *i.e.*, “decoupling cRT” and “decoupling LWS” [5] which decouple the feature learning and classifier calibration process.

2) *Results*: The experiment results are reported in Table I, II, III for CIFAR-10/100-LT, ImageNet-LT and iNaturalist, respectively. Generally, we have three observations. Firstly, all the long-tailed recognition methods including ours outperform the cross-entropy baseline. For example, our method outperforms the cross-entropy baseline by 3.6% on ImageNet-LT. Secondly, our method can be combined with previous works to improve the accuracy further. For example, despite the strong baseline provided by RIDE and BCL, combining with our method yields obvious improvement by around 1.4% and 1.0% on ImageNet-LT, respectively. Last but not least, we observe that it is hard for most of the previous long-tailed recognition methods to strike a balance between the accuracy of “Head” group and that of “Tail” group. For example, comparing Balanced Softmax to Cross Entropy on iNaturalist 2018, the accuracy increases by about 10% for the Tail group, while for the Head group, the accuracy decreases by more than 6%. However, in the majority of cases, our method can bring consistent improvements on all the three groups.

### D. Ablation Studies

1) *Existence of Rare-Specific and Frequent-Specific Features*: To clarify the concept of class-specific feature channels, we provide extra experiments to elaborate on it. To test the different effects of each channel on rare and frequent classes, we mask each feature channel and record the performance drop with respect to rare classes and frequent classes. If masking a feature channel results in a severe performance drop for rare classes but a negligible drop for frequent classes, this feature channel is much more important for rare classes than for frequent classes and can be considered as the “rare-specific” channel. The “frequent-specific” channel can be determined in a similar way. Table V shows the tail and head classes’ performance when the rare-specific and frequent-specific channels are masked. We observe that different feature channels do have different effects on rare and frequent classes. For example, when masking rare-specific channels, the accuracy of tail classes drops heavily (1.0%), while the

TABLE V

ABLATION STUDY ON THE PERFORMANCE VARIATION WHEN THE RARE-SPECIFIC, FREQUENT-SPECIFIC, AND COMMON FEATURE CHANNELS ARE MASKED, RESPECTIVELY. EXPERIMENTS ARE CONDUCTED WITH RESNET-50 ON IMAGENET-LT

| Method                          | All  | Tail       | Head        |
|---------------------------------|------|------------|-------------|
| ResNet-50                       | 42.2 | 6.3        | 64.2        |
| mask rare-specific channels     | 41.1 | 5.3 (-1.0) | 64.1 (-0.1) |
| mask frequent-specific channels | 41.4 | 6.1 (-0.2) | 62.9 (-1.3) |
| mask common channels            | 37.1 | 3.5 (-2.8) | 58.1 (-6.1) |

accuracy of head classes keeps nearly unchanged. Besides, the channels which are important for both rare and frequent classes are named “common” channels. From Table V, we observe that masking those common channels may result in obvious accuracy drop for both tail and head classes.

The existence of rare-specific and frequent-specific features further justifies the motivation of NBN: Our NBN aims to make  $\gamma$  and  $\beta$  of BN more evenly distributed across channels. While  $\gamma$  and  $\beta$  represent the strength of a feature channel statistically, more evenly distributed  $\gamma$  and  $\beta$  of BN means the strengths of feature channels are comparable, *i.e.*, the strengths of rare-specific and frequent-specific feature channels are comparable, which may reduce the feature bias and benefit the model’s long-tailed recognition performance.

2) *Comparison With Variance Regularization*: As shown in Table IV, we compare our NBN method with the variance regularization which also encourages more balanced parameters of BN layer. It can be seen that both variance regularization and our method outperform the cross-entropy baseline remarkably, which verifies that balancing the parameters of BN layer may reduce the feature bias and thus benefit the long-tailed recognition performance. We also observe that our NBN shows superior performance than variance regularization. As discussed in Sec. Methods, it may be because our NBN strikes a balance between balancing the parameters and fitting the data. In contrast, stronger variance regularization leads to inferior results, implying variance regularization may conflict with the data-fitting objective.

3) *Combination With Decoupling cRT*: We combine our NBN with the prevailing two-stage long-tailed recognition method, *i.e.*, decoupling cRT [5], where the backbone and the classifier are jointly trained with instance-balanced sampling in stage one to learn the visual representation, and in stage two the classifier is calibrated via a re-sampling strategy with the backbone fixed. We conduct the experiment on ImageNet-LT [15] and iNaturalist 2018 [16]. As shown in Table VIII, with the help of NBN rectifying the feature bias, the accuracy is remarkably improved, *e.g.*, around 2% improvement on ImageNet-LT. Moreover, we empirically find that the learning of magnitude in stage two can further improve the performance by about 1% on both datasets. The results reflect that our method can effectively rectify the bias of features and improve the generalization ability of features beyond instance-balanced sampling.

4) *Effect of Different Positions to Insert NBN*: In Sec. Methods, we introduce that we apply NBN to the last stage of ResNet architecture. There are two alternative ways to

TABLE VI

COMPARISON OF DIFFERENT POSITIONS TO INSERT NBN. “TYPE A” MEANS WE REPLACE ALL THE BN LAYERS IN THE LAST STAGE OF RESNET ARCHITECTURE WITH NBN LAYER, WHILE “TYPE B” MEANS WE EMPLOY NBN AT POSITIONS WHICH ARE COMPLEMENTARY TO “OURS”. “TYPE C” MEANS WE REPLACE ALL BN LAYERS WITH NBN. THE EXPERIMENTS ARE CONDUCTED WITH RESNET-32 ON CIFAR-10 AND RESNET-50 ON IMAGENET-LT

| Datasets    | Normalization type | All         | Tail        | Medium      | Head        |
|-------------|--------------------|-------------|-------------|-------------|-------------|
| CIFAR-10-LT | Baseline           | 78.0        | -           | 65.7        | 81.1        |
|             | Type A             | 80.0        | -           | 69.4        | 82.6        |
|             | Type B             | 78.6        | -           | 69.3        | 80.9        |
|             | Type C             | <b>80.3</b> | -           | 68.8        | <b>83.2</b> |
|             | Ours               | 80.0        | -           | <b>71.6</b> | 82.1        |
| ImageNet-LT | Baseline           | 42.2        | 6.3         | 34.7        | 64.2        |
|             | Type A             | 47.0        | <b>13.7</b> | 40.4        | 66.9        |
|             | Type B             | 43.0        | 6.8         | 35.8        | 64.9        |
|             | Type C             | 46.9        | 12.9        | 40.2        | <b>67.1</b> |
|             | Ours               | <b>47.3</b> | 13.4        | <b>40.9</b> | <b>67.1</b> |

TABLE VII

EFFECT OF SHARING  $g$  ACROSS NBN LAYERS. EXPERIMENTS ARE CONDUCTED ON IMAGENET-LT WITH RESNET-50

|                    | All         | Tail        | Medium      | Head        |
|--------------------|-------------|-------------|-------------|-------------|
| cross entropy      | 42.2        | 6.3         | 34.7        | 64.2        |
| not sharing        | 45.2        | 8.8         | 38.0        | 66.9        |
| partially sharing  | 46.1        | 10.8        | 39.3        | 67.0        |
| sharing all (Ours) | <b>47.3</b> | <b>13.4</b> | <b>40.9</b> | <b>67.1</b> |

TABLE VIII

ACCURACY ON IMAGENET-LT, AND INATURALIST OF DECOUPLING CRT [5] WITH OR WITHOUT NBN. FOLLOWING [5], IN STAGE ONE THE BACKBONE AND THE CLASSIFIER ARE JOINTLY TRAINED FOR 90 EPOCHS, WHILE IN STAGE TWO ONLY THE CLASSIFIER IS TUNED FOR 10 OR 30 EPOCHS FOR IMAGENET-LT AND INATURALIST, RESPECTIVELY. THE “NBN” MEANS WE EMPLOY OUR NBN IN STAGE ONE AND KEEP THE PARAMETERS OF NBN FIXED IN STAGE TWO. THE “NBN\*” DENOTES THAT THE MAGNITUDE  $g$  OF NBN IS ALSO UPDATED IN STAGE TWO

| Datasets    | Stage   | Methods  | All         | Tail        | Medium      | Head        |
|-------------|---------|----------|-------------|-------------|-------------|-------------|
| ImageNet-LT | Stage 1 | Baseline | 42.2        | 6.3         | 34.7        | 64.2        |
|             |         | NBN      | <b>47.3</b> | <b>13.4</b> | <b>40.9</b> | <b>67.1</b> |
|             | Stage 2 | Baseline | 47.4        | 26.2        | 43.8        | 59.3        |
|             |         | NBN      | 49.9        | 24.1        | 46.2        | <b>63.5</b> |
|             |         | NBN*     | <b>50.7</b> | <b>26.8</b> | <b>47.4</b> | 63.3        |
| iNaturalist | Stage 1 | Baseline | 62.3        | 56.7        | <b>64.3</b> | <b>74.3</b> |
|             |         | NBN      | <b>63.3</b> | <b>59.3</b> | <b>64.3</b> | 73.4        |
|             | Stage 2 | Baseline | 65.2        | 63.2        | 66.0        | 69.0        |
|             |         | NBN      | 65.9        | 63.3        | 66.8        | <b>71.4</b> |
|             |         | NBN*     | <b>66.9</b> | <b>64.8</b> | <b>67.9</b> | 70.4        |

insert NBN layer. In Table VI, “Type A” means we replace all the BN layers in the last stage with NBN layer, while “Type B” means we employ NBN at positions which are complementary to ours. “Type C” means we replace all BN layers with NBN. We observe that “Type A” and “Type C” achieve comparable results while “Type B” is inferior to ours. All the results show that the positions we select to employ NBN are sufficient (by comparing to “Type A” and “Type C”) and necessary (by comparing to “Type B”) to reduce feature bias.

TABLE IX

ABLATION STUDY ON THE RESPECTIVE EFFECT OF LEARNABLE PARAMETERS  $g$  AND DECOUPLING WITHOUT PERFORMING NORMALIZATION. THE EXPERIMENT IS PERFORMED WITH RESNET-50 ON IMAGENET-LT

|         | Cross Entropy | Ours        | Fix $g$ | Ours w/o. normalization |
|---------|---------------|-------------|---------|-------------------------|
| Acc (%) | 42.2          | <b>47.3</b> | 17.3    | 44.9                    |

5) *Effect of Sharing Magnitude  $g$  Across Different NBN Layers*: As the feature fed into the classifier is a summation of outputs from the main branch and the residual connections, we share the magnitude  $g$  across all the NBN layers to ensure these outputs have comparable strength. As shown in Table VII, we compare our way (*i.e.*, “sharing all”) to two alternative ways: “not sharing” which means each NBN layer independently owns one magnitude  $g$  and “partially sharing” which shares  $g$  only within each residual block. We observe that both “partially sharing” and ours perform better than the “not sharing” way, and ours is the best of all.

6) *Effect of Learnable Magnitude  $g$  and Normalization*: To investigate the respective effect of learnable magnitude  $g$  and the normalization operation, we compare NBN (“Ours”) with the one that fixes  $g$  throughout the training process, and the one without normalization operation in Table IX. We observe that fixing  $g$  during training performs severely worse than our NBN, and even worse than the cross-entropy baseline, which shows learning  $g$  is important for fitting the data to maintain the model capacity. Moreover, if we remove normalization from our NBN (“Ours w/o. normalization”), the accuracy remains higher than the cross-entropy baseline but is much worse than NBN, which verifies the effectiveness of performing normalization in NBN.

7) *NBN Versus WN [37]*: In Sec. Methods, we discuss the relationship between NBN and WN. From the operation-level, WN is distinct from NBN as WN is applied to the convolutional layers while NBN is applied to BN layers. In Table X, we compare NBN with WN. We conduct the experiments on CIFAR-10-LT with ResNet-32 and ImageNet-LT with ResNet-50. For fairness, all the settings are kept the same except for the type of layer to employ the parameter normalization operation. We observe that WN doesn’t achieve obvious improvement compared to the baseline, while our NBN performs remarkably better than WN. This comparison shows that the original WN cannot deal with the feature bias problem in long-tailed recognition.

8) *Normalizing BN Versus Normalizing Features*: We have also compared our proposed NBN with normalizing features. Given the feature shape output by BN is  $[N, C, H, W]$ , we try two normalization ways which normalize features across the dimensions of  $[C]$  and  $[C, H, W]$  respectively. The results listed in Table XII show that feature normalization along the channel  $[C]$  axis also brings performance gain compared to the original training (about 2%), but less than that achieved by NBN (about 5%). The difference between NBN and feature normalization is NBN performs normalization in a dataset level (as it normalizes the statistics across channels), while feature normalization performs normalization in a sample level

TABLE X

COMPARISONS BETWEEN NBN AND WN [37]. THE EXPERIMENT IS CONDUCTED WITH RESNET-32 ON CIFAR-10-LT AND RESNET-50 ON IMAGENET-LT. CROSS ENTROPY DENOTES TRAINING WITH CROSS-ENTROPY LOSS ONLY

| Datasets    | Positions     | All         | Tail        | Medium      | Head        |
|-------------|---------------|-------------|-------------|-------------|-------------|
| CIFAR-10-LT | Cross Entropy | 78.0        | -           | 65.7        | 81.1        |
|             | NBN           | <b>80.0</b> | -           | <b>69.4</b> | <b>82.6</b> |
|             | WN            | 78.3        | -           | 65.5        | 81.5        |
| ImageNet-LT | Cross Entropy | 42.2        | 6.3         | 34.7        | 64.2        |
|             | NBN           | <b>47.3</b> | <b>13.4</b> | <b>40.9</b> | <b>67.1</b> |
|             | WN            | 42.0        | 6.9         | 34.4        | 63.8        |

TABLE XI

PERFORMANCE COMPARISON BETWEEN CROSS-ENTROPY BASELINE AND OURS ON BALANCED DATASETS

| Model         | ResNet-32   |           | ResNet-50   |
|---------------|-------------|-----------|-------------|
| Dataset       | CIFAR-10    | CIFAR-100 | ImageNet    |
| Cross Entropy | <b>93.8</b> | 70.5      | 75.3        |
| Ours          | 93.4        | 70.5      | <b>75.7</b> |

TABLE XII

COMPARISON WITH FEATURE NORMALIZATION, LN AND IN WITH RESNET-50 ON IMAGENET-LT

| ImageNet-LT                                     | All         | Tail        | Medium      | Head        |
|-------------------------------------------------|-------------|-------------|-------------|-------------|
| ResNet-50                                       | 42.2        | 6.3         | 34.7        | 64.2        |
| ResNet-50 + feature norm on channel $[C]$       | 44.5        | 10.4        | 37.8        | 65.0        |
| ResNet-50 + feature norm on channel $[C, H, W]$ | 39.8        | 0.9         | 28.9        | <b>67.1</b> |
| ResNet-50 + Ours                                | <b>47.3</b> | <b>13.4</b> | <b>40.9</b> | <b>67.1</b> |

TABLE XIII

EXPERIMENTS ON REPLACING/ADDING LN OR IN IN RESNETS

| ImageNet-LT                    | All         | Tail        | Medium      | Head        |
|--------------------------------|-------------|-------------|-------------|-------------|
| ResNet-50                      | 42.2        | 6.3         | 34.7        | 64.2        |
| ResNet-50 (replace BN with LN) | 32.5        | 4.9         | 24.5        | 52.3        |
| ResNet-50 (replace BN with IN) | 27.9        | 0.1         | 15.4        | 53.4        |
| ResNet-50+LN                   | 43.6        | 11.5        | 36.9        | 63.3        |
| ResNet-50+IN                   | 27.8        | 0.2         | 15.1        | 53.3        |
| ResNet-50 + Ours               | <b>47.3</b> | <b>13.4</b> | <b>40.9</b> | <b>67.1</b> |

(as it normalizes the features for each sample). We think normalization in a dataset level is more reasonable and effective.

9) *Comparison With LayerNorm and Instance Norm*:

To further demonstrate the superiority of our method, we make comparison with other normalization techniques, LayerNorm (LN) and InstanceNorm (IN), under long-tailed recognition setting. In our experiment, we replace NBN with LN / IN, or add extra LN / IN layers after the BN layers. As demonstrated in Table XIII, compared to the ResNet-50 baseline, replacing BN layers with LN or IN leads to a significant performance drop, while inserting extra LN layers yields improvement (+1.4%). However, our NBN achieves more than 5% performance gain (43.6% v.s. 47.3%). All those results verify the superiority of our NBN compared to LN and IN.

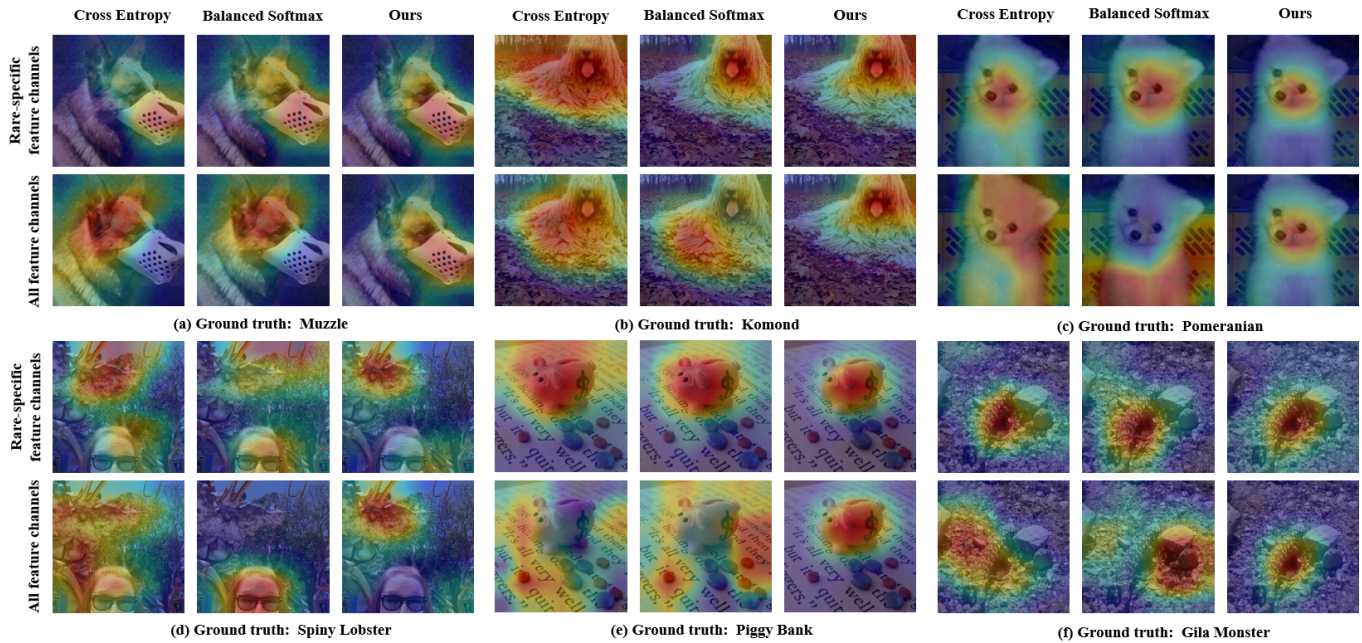


Fig. 4. Visualization of the rare-specific features channels and combination of all the feature channels with the vanilla ResNet-50 trained with cross-entropy loss (**Cross Entropy**) and Balanced Softmax (**Balanced Softmax**), and ResNet-50 equipped with NBN trained with cross-entropy loss (**Ours**). We observe that with the aid of NBN, the model keeps focusing on the class-discriminative regions.

TABLE XIV  
RESULTS OF APPLYING OUR METHOD TO LONG-TAILED  
DETECTION AND SEGMENTATION ON LVIS-V1

| 2x schedule          | mAP         | Detection   |             |             | mAP         | Segmentation |             |             |
|----------------------|-------------|-------------|-------------|-------------|-------------|--------------|-------------|-------------|
|                      |             | APr         | APc         | APf         |             | APr          | APc         | APf         |
| Cross Entropy        | 19.9        | 1.3         | 16.7        | 29.3        | 19.2        | 0.0          | 17.2        | 29.5        |
| EQL                  | 22.0        | 10.7        | 18.8        | 30.6        | 22.7        | 15.1         | 20.4        | 28.6        |
| Seesaw Loss          | 25.3        | 12.7        | 23.7        | <b>32.6</b> | 24.7        | 14.9         | 23.9        | 29.8        |
| Cross Entropy + Ours | 23.5        | 9.2         | 21.0        | 32.4        | 22.5        | 10.3         | 21.0        | 29.6        |
| Seesaw Loss + Ours   | <b>26.0</b> | <b>14.4</b> | <b>24.5</b> | <b>32.6</b> | <b>25.3</b> | <b>16.1</b>  | <b>25.8</b> | <b>29.9</b> |

10) *Effect of NBN on LVIS-V1*: We also extend our method to long-tailed detection and segmentation tasks on LVIS-V1. All the experiments are conducted with MMDetection and follow a similar training strategy with Seesaw method [53] for fair comparison, except that the batch size is set to 4 and the learning rate is set to 0.005 following the linear rule [54]. Following [53], we adopt a normalized linear classification head for category prediction of proposals. For evaluation, we report the detection and segmentation results with mAP, APr, APc, and APf, which are commonly used metrics to evaluate the average performance, performance on rare, common, and frequent groups respectively. As shown in Table XIV, compared to Seesaw, our method increases both the detection and segmentation performance, with 0.7% and 0.6% overall gain respectively. Moreover, considering APr, our method surpasses Seesaw by 1.7% and 1.2%. The results generally demonstrate our method also benefits the long-tailed detection and segmentation performance. As our method is not specifically designed for long-tailed detection and segmentation tasks, how to make NBN more suitable for detection and segmentation tasks and improve the performance further remains future work.

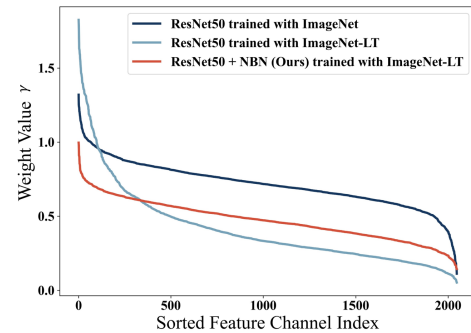


Fig. 5. Comparison of the weight of the last BN layer among the ResNet-50 trained on ImageNet, on ImageNet-LT, and ResNet-50 with NBN on ImageNet-LT.

11) *Effect of NBN With Balanced Dataset*: Technically, our NBN can also be adopted when the training set is balanced. As shown in Table XI, we employ NBN on original CIFAR-10, CIFAR-100, and ImageNet datasets which are balanced. We observe that when the dataset is balanced, our method achieves comparable results to the cross-entropy baseline. It is because our method is designed to mitigate the feature bias caused by the long-tailed distribution of training samples, and when the dataset is balanced, such feature bias doesn't exist, rendering our method ineffective.

12) *Effect of NBN and Logit Rectification (LR)*: In Table XV, we verify the effectiveness of our proposed NBN and logit rectification (LR) module. With progressively adding NBN and LR into the architecture, the accuracy of our method is consistently improved, which validates the effectiveness of both NBN and LR. Specifically, the final performance improvement can be largely attributed to the adoption of NBN.

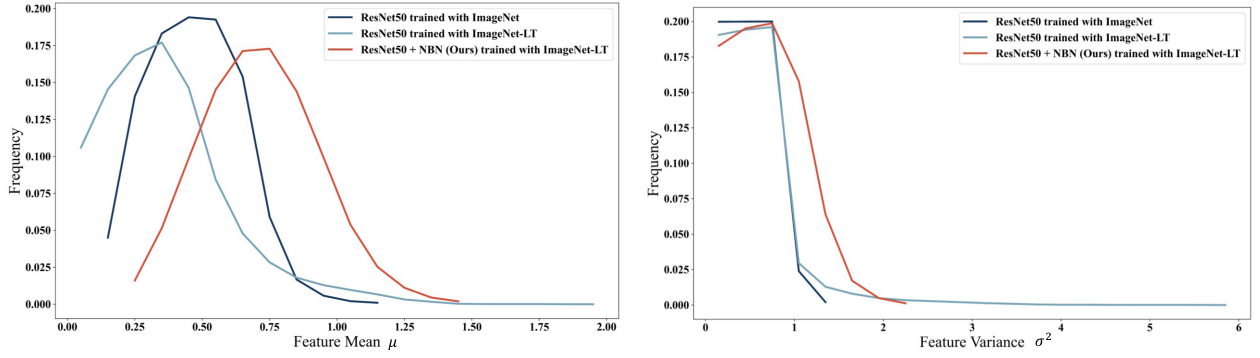


Fig. 6. Histogram of feature statistics across channels. The comparison is among the ResNet-50 trained on ImageNet, ImageNet-LT, and ResNet-50 with NBN on ImageNet-LT.

TABLE XV

ABLATIONS FOR THE EFFECT OF NBN AND LOGIT RECTIFICATION (LR) WITH RESNET-50 ON IMAGENET-LT

|                  | Baseline | +NBN        | +NBN+LR     |
|------------------|----------|-------------|-------------|
| Cross Entropy    | 42.2     | 47.3 (+5.1) | 48.8 (+6.6) |
| Balanced Softmax | 48.8     | 51.5 (+2.7) | 53.0 (+4.2) |

TABLE XVI

COMPARISON OF THE VARIANCE OF FEATURE STATISTICS (MEAN  $\mu$  AND VARIANCE  $\sigma$ ). THE EXPERIMENTS ARE CONDUCTED ON IMAGENET-LT WITH RESNET-50 BACKBONE. THE FEATURES THAT ARE FED INTO THE CLASSIFIER ARE INVESTIGATED

|               | Variance of $\mu$ | Variance of $\sigma$ |
|---------------|-------------------|----------------------|
| Cross Entropy | 0.040             | 0.152                |
| Ours          | 0.028             | 0.074                |

As discussed in Sec. Methods, the LR module may rectify the output statistics and thus improve the accuracy further.

13) *Effect of NBN on Rectifying Feature Bias*: To demonstrate the rectification effect of NBN, we select three models, ResNet-50 trained on ImageNet, ResNet-50 trained on ImageNet-LT, and ResNet-50 equipped with NBN trained on ImageNet-LT. As shown in Fig. 5, the y-axis represents the Weight value (*i.e.*,  $\gamma$ ) of each channel in the last BN layer of ResNet-50, and the x-axis represents the sorted feature channel index. The feature channels are sorted in the descending order of  $\gamma_k, k \in [0, 2047]$  where  $k$  is the channel index. It is obvious that the weight curve for the baseline model trained on ImageNet-LT is quite imbalanced. In contrast, the curve corresponding to the one trained on ImageNet is more flat. With the aid of NBN, the skewed curve is flattened and shows similar slope to the one trained on ImageNet. We additionally plot the histogram of feature statistics in Fig. 6. We collect the feature embedding of samples from the ImageNet-LT test set and calculate the mean  $\mu$  and variance  $\sigma$  for each feature channel with ResNet-50 trained on ImageNet (the dark blue curve), ResNet-50 trained on ImageNet-LT (the light blue curve), and ResNet-50 inserted with NBN trained on ImageNet-LT (the red curve) respectively. It can be seen that there is a long tail in the histogram of ResNet-50

TABLE XVII

COMPARISON BETWEEN LOGN AND LR WITH RESNEXT-50 ON IMAGENET-LT

| Method                    | All         | Tail        | Medium      | Head        |
|---------------------------|-------------|-------------|-------------|-------------|
| Cross Entropy             | 45.8        | 9.0         | 39.0        | <b>67.2</b> |
| Balanced Softmax          | 51.1        | 30.2        | 48.1        | 62.3        |
| LogN [49]                 | 51.6        | 35.0        | 50.3        | 59.1        |
| Balanced Softmax + LogN   | 49.6        | 32.7        | 46.6        | 59.3        |
| Balanced Softmax+NBN+LogN | 49.9        | <b>37.4</b> | 46.9        | 58.2        |
| Balanced Softmax+NBN+LR   | <b>53.8</b> | 33.4        | <b>50.9</b> | 64.6        |

trained on ImageNet-LT compared to the ResNet-50 trained on ImageNet, which means the variance of  $\mu$  or  $\sigma$  across channels is much larger for ResNet-50 trained on ImageNet-LT than for ResNet-50 trained on ImageNet. With the rectification of NBN, the issue is largely alleviated. We also provide the variance of  $\mu$  and the variance of  $\sigma$  in Table XVI, which are calculated as follows

$$\text{Var}(\mu) = \frac{1}{C-1} \sum_{k=1}^C [\mu_k - \frac{1}{C} \sum_{k=1}^C \mu_k] \quad (5)$$

$$\text{Var}(\sigma) = \frac{1}{C-1} \sum_{k=1}^C [\sigma_k - \frac{1}{C} \sum_{k=1}^C \sigma_k]. \quad (6)$$

As shown in Table XVI, the variance of  $\mu$  or  $\sigma$  is much larger for the cross-entropy baseline than for our method, which is consistent with what we observe in Fig. 6.

14) *Comparison Between LR and LogN [49]*: Both LR and LogN propose to normalize the logits under long-tailed learning scenario. The main difference between the two approaches is that LR is inserted in the network training phase and affects the gradient back-propagation, while LogN is a kind of post-processing method which does not engage in the training. In Table XVII, we provide the experimental results of ResNext-50 with LR and LogN on ImageNet-LT. We observe that both LogN and LR improve the long-tailed recognition performance. Our method (the last row of table) outperforms the results reported by LogN (the third row) by more than 2%. By replacing LR with LogN in our solution (see the penultimate row), the accuracy decreases significantly, which shows that LR is more compatible with our designed NBN than LogN.

## E. Visualization

To verify that NBN is effective on enhancing the strength of rare-specific features, we visualize the attention [7] corresponding to rare-specific feature channels and that corresponding to all feature channels. Specifically, by examining the classifier weights of each category, we select the feature channels which are uniquely important for rare classes. By aggregating the activation maps of corresponding channels, the image area related to rare-specific features is highlighted, as shown in Fig. 4. By comparing with the attentions of Cross Entropy and Balanced Softmax, we observe that although the rare-specific features have already emerged and captured the discriminative part, their weak strength makes the overall attention unexpectedly focus on irrelevant regions, which explains why the final prediction is incorrect. In contrast, our method enables the model to keep focusing on the discriminative features. The visualization results validate our assumption and motivation that the rare-specific features are inherently weak, rendering the model biased towards frequent classes.

## V. CONCLUSION

In this paper, we observe that under long-tailed scenarios, the learned features may be strongly biased towards the frequent classes and its distribution may exhibit unexpected imbalance, in the sense that the strengths of discriminative features for rare classes are weaker than those for frequent classes. To address this issue, we introduce a simple yet effective strategy, NBN, to explicitly rectify the feature bias under long-tailed scenarios. Through extensive experimental results, we demonstrate that our method is plug-and-play and brings consistent improvements compared to previous methods, achieving new state-of-the-arts.

## REFERENCES

- [1] O. Russakovsky et al., "ImageNet large scale visual recognition challenge," *Int. J. Comput. Vis.*, vol. 115, no. 3, pp. 211–252, Dec. 2015.
- [2] T.-Y. Lin et al., "Microsoft COCO: Common objects in context," in *Proc. ECCV*, Jan. 2014, pp. 740–755.
- [3] K. Cao, C. Wei, A. Gaidon, N. Aréchiga, and T. Ma, "Learning imbalanced datasets with label-distribution-aware margin loss," in *Proc. NeurIPS*, Jan. 2019, pp. 1565–1576.
- [4] Y. Cui, M. Jia, T.-Y. Lin, Y. Song, and S. Belongie, "Class-balanced loss based on effective number of samples," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR)*, Jun. 2019, pp. 9260–9269.
- [5] B. Kang et al., "Decoupling representation and classifier for long-tailed recognition," in *Proc. ICLR*, Jan. 2019, pp. 1–12.
- [6] S. Alshammari, Y.-X. Wang, D. Ramanan, and S. Kong, "Long-tailed recognition via weight balancing," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR)*, Jun. 2022, pp. 6887–6897.
- [7] S. Zagoruyko and N. Komodakis, "Paying more attention to attention: Improving the performance of convolutional neural networks via attention transfer," in *Proc. ICLR*, Jan. 2016, pp. 1–13.
- [8] C. Drummond et al., "C4.5, class imbalance, and cost sensitivity: Why under-sampling beats over-sampling," in *Proc. Workshop Learn. From Imbalanced Datasets II*, 2003, pp. 1–8.
- [9] N. Japkowicz and S. Stephen, "The class imbalance problem: A systematic study," *Intell. Data Anal.*, vol. 6, pp. 429–449, Oct. 2002.
- [10] N. V. Chawla, K. W. Bowyer, L. O. Hall, and W. P. Kegelmeyer, "SMOTE: Synthetic minority over-sampling technique," *J. Artif. Intell. Res.*, vol. 16, pp. 321–357, Jun. 2002.
- [11] S. Park, Y. Hong, B. Heo, S. Yun, and J. Y. Choi, "The majority can help the minority: Context-rich minority oversampling for long-tailed classification," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR)*, Jun. 2022, pp. 6877–6886.
- [12] J. Tan et al., "Equalization loss for long-tailed object recognition," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR)*, Jun. 2020, pp. 11659–11668.
- [13] J. Ren et al., "Balanced meta-softmax for long-tailed visual recognition," in *Proc. NeurIPS*, 2020, pp. 4175–4186.
- [14] A. Krizhevsky et al., "Learning multiple layers of features from tiny images," Univ. Toronto, Toronto, ON, Canada, Tech. Rep. 1, 2009.
- [15] Z. Liu, Z. Miao, X. Zhan, J. Wang, B. Gong, and S. X. Yu, "Large-scale long-tailed recognition in an open world," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR)*, Jun. 2019, pp. 2532–2541.
- [16] G. Van Horn et al., "The iNaturalist species classification and detection dataset," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit.*, Jun. 2018, pp. 8769–8778.
- [17] M. Kubát and S. Matwin, "Addressing the curse of imbalanced training sets: one-sided selection," in *Proc. ICML*, Jan. 1997, pp. 179–186.
- [18] H. He and E. A. Garcia, "Learning from imbalanced data," *IEEE Trans. Knowl. Data Eng.*, vol. 21, no. 9, pp. 1263–1284, Sep. 2009.
- [19] T.-Y. Lin, P. Goyal, R. Girshick, K. He, and P. Dollár, "Focal loss for dense object detection," in *Proc. IEEE Int. Conf. Comput. Vis. (ICCV)*, Oct. 2017, pp. 2999–3007.
- [20] S. Zhang, Z. Li, S. Yan, X. He, and J. Sun, "Distribution alignment: A unified framework for long-tail visual recognition," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR)*, Jun. 2021, pp. 2361–2370.
- [21] X. Wang, L. Lian, Z. Miao, Z. Liu, and S. X. Yu, "Long-tailed recognition by routing diverse distribution-aware experts," in *Proc. ICLR*, Jan. 2020, pp. 1–15.
- [22] B. Li, Z. Han, H. Li, H. Fu, and C. Zhang, "Trustworthy long-tailed classification," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR)*, Jun. 2022, pp. 6960–6969.
- [23] J. Van Hulse, T. M. Khoshgoftaar, and A. Napolitano, "Experimental perspectives on learning from imbalanced data," in *Proc. 24th Int. Conf. Mach. Learn.*, Jun. 2007, pp. 935–942.
- [24] S. Ando and C. Huang, "Deep over-sampling framework for classifying imbalanced data," in *Proc. Joint Eur. Conf. Mach. Learn. Knowl. Discovery Databases*, Skopje, Macedonia, Jan. 2017, pp. 770–785.
- [25] P. Chu, X. Bian, S. Liu, and H. Ling, "Feature space augmentation for long-tailed data," in *Proc. ECCV*, Jan. 2020, pp. 694–710.
- [26] S. Li, K. Gong, C. H. Liu, Y. Wang, F. Qiao, and X. Cheng, "MetaSAug: Meta semantic augmentation for long-tailed visual recognition," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR)*, Jun. 2021, pp. 5208–5217.
- [27] Z. Zhong, J. Cui, S. Liu, and J. Jia, "Improving calibration for long-tailed recognition," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR)*, Jun. 2021, pp. 16484–16493.
- [28] H. Zhang, M. Cissé, Y. Dauphin, and D. López-Paz, "Mixup: Beyond empirical risk minimization," in *Proc. ICLR*, Jan. 2017, pp. 1–13.
- [29] T. DeVries and G. W. Taylor, "Improved regularization of convolutional neural networks with cutout," 2017, *arXiv:1708.04552*.
- [30] Z. Zhong, L. Zheng, S. Li, and Y. Yang, "CamStyle: A novel data augmentation method for person re-identification," *IEEE Trans. Image Process.*, vol. 28, no. 3, pp. 1176–1190, Mar. 2019.
- [31] Z. Yang et al., "Small object augmentation of urban scenes for real-time semantic segmentation," *IEEE Trans. Image Process.*, vol. 29, pp. 5175–5190, 2020.
- [32] M. Li, Y.-M. Cheung, and Y. Lu, "Long-tailed visual recognition via Gaussian clouded logit adjustment," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR)*, Jun. 2022, pp. 6919–6928.
- [33] S. Ioffe and C. Szegedy, "Batch normalization: Accelerating deep network training by reducing internal covariate shift," in *Proc. Int. Conf. Mach. Learn.*, Jan. 2015, pp. 448–456.
- [34] X. Wang, Y. Jin, M. Long, J. Wang, and M. I. Jordan, "Transferable normalization: Towards improving transferability of deep neural networks," in *Proc. NeurIPS*, vol. 32, Sep. 2019, pp. 1951–1961.
- [35] Y. Li, N. Wang, J. Shi, J. Liu, and X. Hou, "Revisiting batch normalization for practical domain adaptation," in *Proc. ICLR*, Jan. 2016, pp. 1–12.
- [36] L. Cheng, C. Fang, D. Zhang, G. Li, and G. Huang, "Compound batch normalization for long-tailed image classification," in *Proc. 30th ACM Int. Conf. Multimedia*, Oct. 2022, pp. 1925–1934.

- [37] T. Salimans and D. P. Kingma, "Weight normalization: A simple reparameterization to accelerate training of deep neural networks," in *Proc. NeurIPS*, vol. 29, Dec. 2016, pp. 901–909.
  - [38] M. A. Jamal, M. Brown, M.-H. Yang, L. Wang, and B. Gong, "Rethinking class-balanced methods for long-tailed visual recognition from a domain adaptation perspective," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR)*, Jun. 2020, pp. 7607–7616.
  - [39] J. Cui, Z. Zhong, S. Liu, B. Yu, and J. Jia, "Parametric contrastive learning," in *Proc. IEEE/CVF Int. Conf. Comput. Vis. (ICCV)*, Oct. 2021, pp. 695–704.
  - [40] J. Cai, Y. Wang, and J.-N. Hwang, "ACE: Ally complementary experts for solving long-tailed recognition in one-shot," in *Proc. IEEE/CVF Int. Conf. Comput. Vis. (ICCV)*, Oct. 2021, pp. 112–121.
  - [41] Y. Zhao, W. Chen, X. Tan, K. Huang, and J. Zhu, "Adaptive logit adjustment loss for long-tailed visual recognition," in *Proc. AAAI*, vol. 36, Jun. 2022, pp. 3472–3480.
  - [42] B. Zhu, Y. Niu, X. Hua, and H. Zhang, "Cross-domain empirical risk minimization for unbiased long-tailed classification," in *Proc. AAAI*, Jan. 2021, pp. 3589–3597.
  - [43] X. Yang, Y. Chen, X. Yue, S. Xu, and C. Ma, "T-distributed spherical feature representation for imbalanced classification," in *Proc. AAAI*, vol. 37, Jun. 2023, pp. 10825–10833.
  - [44] Y. Du, J. Shen, X. Zhen, and C. G. M. Snoek, "SuperDisco: Super-class discovery improves visual recognition for the long-tail," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR)*, Jun. 2023, pp. 19944–19954.
  - [45] J. Zhu, Z. Wang, J. Chen, Y. P. Chen, and Y.-G. Jiang, "Balanced contrastive learning for long-tailed visual recognition," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR)*, Jun. 2022, pp. 6898–6907.
  - [46] J. Liu, J. Zhang, Y. Yi, W. Li, C. Zhang, and Y. Sun, "Memory-based jitter: Improving visual recognition on long-tailed data with diversity in memory," in *Proc. AAAI*, Jan. 2020, pp. 1720–1728.
  - [47] Y.-C. Hsu, C.-Y. Hong, M.-S. Lee, D. Geiger, and T.-L. Liu, "ABC-norm regularization for fine-grained and long-tailed image classification," *IEEE Trans. Image Process.*, vol. 32, pp. 3885–3896, 2023.
  - [48] K. P. Alexandridis, S. Luo, A. Nguyen, J. Deng, and S. Zafeiriou, "Inverse image frequency for long-tailed image recognition," *IEEE Trans. Image Process.*, vol. 32, pp. 5721–5736, 2023.
  - [49] L. Zhao, Y. Teng, and L. Wang, "Logit normalization for long-tail object detection," in *Proc. IJCV*, Jan. 2022, pp. 2114–2134.
  - [50] K. He, X. Zhang, S. Ren, and J. Sun, "Deep residual learning for image recognition," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit. (CVPR)*, Jun. 2016, pp. 770–778.
  - [51] S. Xie, R. Girshick, P. Dollár, Z. Tu, and K. He, "Aggregated residual transformations for deep neural networks," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit.*, Aug. 2017, pp. 1492–1500.
  - [52] K. Liu, K. Chen, and K. Jia, "Convolutional fine-grained classification with self-supervised target relation regularization," *IEEE Trans. Image Process.*, vol. 31, pp. 5570–5584, 2022.
  - [53] J. Wang et al., "Seesaw loss for long-tailed instance segmentation," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR)*, Jun. 2021, pp. 9690–9699.
  - [54] P. Goyal et al., "Accurate, large minibatch SGD: Training ImageNet in 1 hour," 2017, *arXiv:1706.02677*.
- Yuxiang Bao** received the M.S. degree from Beihang University in 2024. He is currently working as an Algorithm Engineer at Alibaba Group.
- Guoliang Kang** is currently a Professor at Beihang University.
- Linlin Yang** is a Lecturer at the Communication University of China.
- Xiaoyue Duan** is currently working as an Algorithm Engineer at Tencent.
- Bo Zhao** is an Associate Professor at Shanghai Jiao Tong University.
- Baochang Zhang** (Senior Member, IEEE) is currently a Professor at Beihang University.